

Koyarwa Hub — Architecture & Setup Analysis



Project Overview

Koyarwa Hub is a **Next.js 14 course & ebook platform** for the Nigerian market with:

- User authentication (email/password + Google OAuth)
- Payment processing (Paystack integration)
- Course & ebook management
- Admin dashboard for managing content
- Role-based access control



Database Schema (Prisma + PostgreSQL)

Core Models

User

```
id (cuid, primary key)
|— email (unique)
|— name
|— passwordHash (nullable for Google-only)
```

```
accounts)
├─ image
├─ role (USER | ADMIN)
├─ hasPaid (boolean, default: false)
├─ paymentRef (reference to payment
transaction)
├─ subscriptionDate (when user paid)
├─ Relations: accounts[], sessions[],
payments[], progress[]
├─ timestamps: createdAt, updatedAt
```

Course

```
id (cuid, primary key)
├─ title
├─ description (Text)
├─ thumbnail (image URL)
├─ category (indexed)
├─ videos (JSON array of {title, url (HLS
m3u8), duration})
├─ published (default: true)
├─ Relations: progress[] (CourseProgress)
├─ timestamps: createdAt, updatedAt
├─ Index: category
```

CourseProgress

```
id (cuid, primary key)
├─ userId (foreign key)
├─ courseId (foreign key)
├─ completedVideos (JSON array of completed
video indexes)
├─ updatedAt
```

└─ Unique constraint: userId + courseId (one progress per user per course)

Ebook

```
id (cuid, primary key)
└─ title
└─ description (Text)
└─ coverImage
└─ category (default: "General", indexed)
└─ pdfUrl (URL to PDF file)
└─ published (default: true)
└─ timestamps: createdAt, updatedAt
└─ Index: category
```

Payment

```
id (cuid, primary key)
└─ userId (foreign key)
└─ amount (in kobo, e.g., 100000 = ₦1,000)
└─ status (PENDING | SUCCESS | FAILED)
└─ reference (unique, Paystack reference)
└─ createdAt
└─ Relations: user
```

NextAuth Models (Adapter-generated)

- **Account:** OAuth provider integration (Google)
 - **Session:** User session tokens
 - **VerificationToken:** Email verification
-

Authentication Flow

Strategy: JWT + NextAuth.js v4

Login Methods

1. Email/Password (CredentialsProvider)

- Password hashed with bcrypt (10 salt rounds)
- Case-insensitive email lookup
- Validates password on login

2. Google OAuth

- Uses Google Cloud OAuth credentials
- Auto-creates user account on first login
- Stores OAuth token in Account model

JWT Token Payload

```
{
  id: user.id,
  email: user.email,
  name: user.name,
  role: user.role,           // USER | ADMIN
  hasPaid: user.hasPaid,    // Boolean
  iat: number,
  exp: number
}
```

Key Features

- **JWT refreshes on every session check** (via `trigger === "update"`)
- Payment status (`hasPaid`) & role updates are reflected **without re-login**
- Admin accounts determined by `ADMIN_EMAILS` env variable at registration time

Protected Routes

Via `middleware.ts`:

```

/dashboard/*      → requires login
/courses/*        → requires login + hasPaid
(or ADMIN role)
/ebooks/*         → requires login + hasPaid
(or ADMIN role)
/payment          → requires login
/admin/*          → requires login + ADMIN
role

```

Payment Flow (Paystack Integration)

Architecture

```

User clicks "Pay"
↓
POST /api/payment/initialize
├─ Check session exists
├─ Check user hasn't paid already
├─ Create Payment record (PENDING)
└─ Call Paystack API → returns

```

```
authorization_url
└─ Return URL to frontend
↓
User redirected to Paystack payment page
↓
Payment gateway processes transaction
↓
Two possible outcomes:

1. WEBHOOK (primary): Paystack →
/api/payment/webhook
├─ Verify HMAC-SHA512 signature
├─ If charge.success event
├─ Update Payment.status → SUCCESS
├─ Update User.hasPaid = true
└─ Send confirmation email

2. POLLING (fallback): User redirected to
/payment/success
├─ Frontend calls POST /api/payment/verify
├─ Verifies with Paystack API
├─ Updates User.hasPaid if successful
└─ Shows success/failure message
```

Key Endpoints

Endpoint	Method	Auth	Purpose
<code>/api/payment/initialize</code>	POST		Start payment, get Paystack URL

Endpoint	Method	Auth	Purpose
<code>/api/payment/webhook</code>	POST	✗	Paystack webhook (verify signature)
<code>/api/payment/verify</code>	POST	✓	Client-side payment verification

Amount Configuration

- **Subscription cost:** `SUBSCRIPTION_AMOUNT_KOBO` env variable
- Default: **100,000 kobo = ₦1,000**



API Routes Structure

Authentication

```
POST    /api/auth/register          Register
email/password account
POST    /api/auth/[...nextauth]/
NextAuth.js routes (signin, signout, session)
```

Courses

```
GET      /api/courses              List
courses (paid users only)
Query params: ?category=X&q=search
```

POST	/api/courses	Create
	course (ADMIN only)	
GET	/api/courses/[id]	Get
	single course (paid users only)	
PUT	/api/courses/[id]	Update
	course (ADMIN only)	
DELETE	/api/courses/[id]	Delete
	course (ADMIN only)	
POST	/api/courses/[id]/progress	Record
	video completion	

Ebooks

GET	/api/ebooks	List
	ebooks (paid users only)	
	Query params: ?category=X&q=search	
POST	/api/ebooks	Create
	ebook (ADMIN only)	
GET	/api/ebooks/[id]	Get
	single ebook (paid users only)	
PUT	/api/ebooks/[id]	Update
	ebook (ADMIN only)	
DELETE	/api/ebooks/[id]	Delete
	ebook (ADMIN only)	

Admin

GET	/api/admin/users	List all
	users (ADMIN only)	
PUT	/api/admin/users/[id]	Update
	user role/payment status	

DELETE /api/admin/users/[id] user	Delete user
--------------------------------------	----------------

Uploads

POST /api/upload media to Cloudinary	Upload
---	--------

Tech Stack & Dependencies

Core Framework

- **Next.js 14.2.15** — App Router, Server Components, API Routes
- **React 18.3.1** — UI library
- **TypeScript 5.5.4** — Type safety

Authentication & Database

- **NextAuth.js 4.24.7** — Auth framework
- **@next-auth/prisma-adapter** — Prisma integration
- **Prisma 5.19.1** — ORM for PostgreSQL
- **PostgreSQL** — Primary database

Security & Passwords

- **bcryptjs 2.4.3** — Hash passwords (10 salt rounds)
- **crypto (built-in)** — HMAC-SHA512 signature verification for webhooks

Payments

- **Paystack API** (via HTTP fetch) – Payment gateway

File Management

- **Cloudinary 2.4.0** – Cloud media hosting (videos, images)

Email

- **Nodemailer 6.9.15** – SMTP email sending

Media Components

- **hls.js 1.5.15** – HLS video streaming player
- **react-pdf 7.7.3** – PDF viewer for ebooks

Styling

- **Tailwind CSS 3.4.10** – Utility CSS
- **tailwind-merge 2.5.2** – Smart class merging
- **clsx 2.1.1** – Conditional classes

Project Structure

```
koyarwa-hub/  
├── prisma/  
│   ├── schema.prisma      # Database  
├── schema  
│   └── seed.ts            # Database  
└── seeding script  
    |
```

```

└─ src/
  └─ app/
    └─ api/                # All API routes
    └─ auth/               # Authentication
    └─ payment/            # Payment
  endpoints
  └─ courses/              # Course CRUD
  └─ ebooks/              # Ebook CRUD
  └─ admin/                # Admin
  operations
  └─ upload/               # Cloudinary
  upload
  └─ (auth)/               # Auth layout
  group
  └─ login/                # Login page
  └─ register/             # Register page
  └─ admin/                # Admin pages
  └─ courses/              # Course pages
  └─ ebooks/              # Ebook pages
  └─ payment/              # Payment pages
  └─ dashboard/            # User dashboard
  └─ layout.tsx            # Root layout
  └─ page.tsx              # Home page
  └─ globals.css           # Global styles
  └─ components/
    └─ Navbar.tsx
    └─ CourseCard.tsx
    └─ EbookCard.tsx
    └─ VideoPlayer.tsx
    └─ PdfReader.tsx
    └─ admin/              # Admin
  components
  └─ ui/                   # Reusable UI
  (Card, Button)
  └─ lib/

```

```

|   |   |— auth.ts           # NextAuth
config
|   |   |— prisma.ts         # Prisma client
singleton
|   |   |— paystack.ts       # Paystack API
client
|   |   |— cloudinary.ts     # Cloudinary
config
|   |   |— mailer.ts         # Email service
|   |   |— utils.ts          # Helper
utilities
|   |   |— i18n.tsx          #
Internationalization
|   |
|   |— types/
|   |   |— next-auth.d.ts    # NextAuth type
extensions
|   |
|   |— middleware.ts         # Route
protection middleware
|
|— package.json
|— tsconfig.json
|— next.config.js
|— tailwind.config.ts
|— postcss.config.js
|— .env.example             # Environment
template
|— README.md

```

Environment Variables

Required

```
# Database
DATABASE_URL=postgresql://...

# NextAuth
NEXTAUTH_URL=http://localhost:3000
NEXTAUTH_SECRET=(32 bytes base64)

# Google OAuth
GOOGLE_CLIENT_ID=...
GOOGLE_CLIENT_SECRET=...

# Paystack
PAYSTACK_SECRET_KEY=sk_test_...
NEXT_PUBLIC_PAYSTACK_PUBLIC_KEY=pk_test_...
PAYSTACK_WEBHOOK_SECRET=...
SUBSCRIPTION_AMOUNT_KOBO=100000

# Cloudinary
CLOUDINARY_CLOUD_NAME=...
CLOUDINARY_API_KEY=...
CLOUDINARY_API_SECRET=...

# Email (SMTP)
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=...
SMTP_PASSWORD=...
EMAIL_FROM=Koyarwa Hub <no-reply@...>

# App
NEXT_PUBLIC_APP_URL=http://localhost:3000
ADMIN_EMAILS=admin@example.com
```



Deployment Checklist

- ☐ PostgreSQL database configured (Railway, Render, Supabase)
 - ☐ Environment variables set in production
 - ☐ `NEXTAUTH_SECRET` generated with `openssl rand -base64 32`
 - ☐ `NEXTAUTH_URL` points to production domain
 - ☐ Google OAuth credentials updated for production domain
 - ☐ Paystack keys switched to **LIVE** (not test)
 - ☐ Paystack webhook URL configured (`/api/payment/webhook`)
 - ☐ Cloudinary API keys secured (not in code)
 - ☐ Email service configured (Gmail app password, Resend, SendGrid, etc.)
 - ☐ Database migrations run: `prisma migrate deploy`
 - ☐ Build tested: `npm run build`
 - ☐ Hosted on Vercel, Railway, or similar
-



Key Implementation Notes

Single-Payment Model

- Users pay **once** (₦1,000) and get **lifetime access** to all courses and ebooks
- No subscription/recurring billing

- `User.hasPaid` is a boolean flag, not a date-based check

Content Publishing

- Courses and Ebooks have a `published` boolean
- Only published content shows to users
- Admin can unpublish without deleting

Admin Role Assignment

- Determined at **registration time** via `ADMIN_EMAILS` env variable
- Admin status cannot be changed after registration
- Must add email to `ADMIN_EMAILS` **before** user registers

Video Streaming

- Videos stored on **Cloudinary** as **HLS (.m3u8)** streams
- Frontend uses **hls.js** library for playback
- Allows adaptive bitrate streaming

Course Progress

- Tracks **completed videos** as a JSON array
- One progress record per user per course (unique constraint)
- No percentage calculation in schema (calculated on frontend)

Payment Safety

- Payment records created **before** redirecting to Paystack

- Webhook signature verified with HMAC-SHA512
- Two update paths: webhook (primary) + verify endpoint (fallback)
- Always returns 200 to webhook to prevent retries